

CardiolA – Ir Além 1

API REST, Verificação de Risco e Disparo de E-mail

19 de maio de 2026

1. Objetivo

O presente relatório descreve a implementação do módulo **Ir Além 1** do projeto CardiolA, que consiste em uma API REST desenvolvida em Python com Flask para recebimento de dados vitais simulados de um ESP32. O sistema realiza verificação de risco em tempo real com base em limiares clínicos de BPM e temperatura corporal, e simula o disparo de alertas por e-mail sempre que uma condição anômala é detectada. O objetivo principal é demonstrar o pipeline completo de coleta, armazenamento, análise e notificação para monitoramento cardíaco remoto.

2. Fluxo Implementado

2.1 Coleta de Dados

A etapa de coleta foi simulada por meio de um script Python que emula o comportamento de um ESP32 real. Foram geradas **30 amostras** contendo os seguintes campos:

- **timestamp**– instante da medição (milissegundos)
- **paciente_id**– identificador do paciente (PAC-0001)
- **temperatura**– temperatura corporal (°C)
- **umidade**– umidade ambiente (%)
- **bpm**– batimentos cardíacos por minuto
- **conectado**– status de conexão do dispositivo (booleano)

Cada amostra foi enviada por meio de uma requisição **HTTP POST** para o endpoint da API, e os dados foram armazenados em memória (lista Python) para consultas posteriores.

2.2 API REST (Flask)

A API foi construída utilizando o microframework Flask e executada em uma thread separada para permitir o funcionamento simultâneo no ambiente Colab. Foram disponibilizados os seguintes endpoints:

- **POST /api/sinais**– recebe um novo registro vital e o armazena
- **GET /api/sinais**– retorna a lista completa de registros
- **GET /api/estatisticas**– calcula médias, máximos e mínimos de BPM e temperatura
- **GET /api/alertas**– lista todos os alertas gerados durante a simulação
- **GET /api/health**– endpoint de verificação de integridade do serviço

Todos os endpoints retornam respostas no formato JSON, com códigos HTTP apropriados (201 para criação, 200 para sucesso).

2.3 Verificação de Risco

A cada nova amostra recebida, um módulo de verificação analisa os valores de BPM e temperatura com base nos seguintes limiares clínicos:

- **BPM > 120!** dispara o alerta **TAQUICARDIA**
- **BPM < 50!** dispara o alerta **BRADICARDIA**
- **Temperatura > 38°C!** dispara o alerta **FEBRE**

Os alertas são registrados em uma lista separada e podem ser consultados pelo endpoint */api/alertas*.

2.4 Disparo de E-mail

Quando um risco é detectado, o sistema simula o envio de um e-mail para o endereço **medico@cardioia.com**. A simulação é feita por meio de saída no console, exibindo:

- Assunto: Ø=b" ALERTA CARDÍACO – PAC-0001
- Corpo: dados do paciente (BPM, temperatura, umidade) e tipo de ocorrência

Essa etapa demonstra a integração do pipeline de alertas com uma possível notificação remota, que futuramente pode ser implementada com protocolos reais de e-mail (*smtplib*).

2.5 Dashboard

Ao final da simulação, os dados armazenados são processados com **Pandas** e **Matplotlib** para gerar as seguintes visualizações:

- Série temporal de **BPM** com limites de alerta destacados
- Série temporal de **temperatura** com linha de febre (38°C)
- Gráfico de dispersão **BPM x Temperatura**, colorido por condição de alerta
- Gráfico de pizza com **proporção de alertas** em relação ao total de amostras

O dashboard permite visualizar rapidamente a ocorrência de anomalias e a evolução dos sinais ao longo do tempo.

3. Tecnologias Utilizadas

As seguintes tecnologias foram empregadas no desenvolvimento deste módulo:

- **Python 3.10**— linguagem principal
- **Flask 3.0**— framework web para construção da API REST
- **requests**— biblioteca cliente HTTP para simulação do ESP32
- **Pandas**— manipulação e análise dos dados
- **Matplotlib**— geração dos gráficos do dashboard
- **Threading**— execução do servidor Flask em background no ambiente Colab

Nota: Todo o código foi executado em Google Colab, o que exigiu o uso de threading para manter o servidor ativo durante a simulação.

4. Resultados

A execução da simulação produziu os seguintes resultados:

- **30 amostras** processadas e armazenadas com sucesso
- **Taxa de alertas** variável conforme a injeção aleatória de anomalias (aproximadamente 10-15% das amostras geraram ao menos um alerta)
- O pipeline completo foi demonstrado de ponta a ponta: coleta !' armazenamento !' verificação !' alerta !' visualização
- Todos os endpoints da API responderam corretamente, com respostas JSON válidas

O relatório confirma a viabilidade técnica da arquitetura proposta para monitoramento cardíaco remoto com capacidade de resposta imediata a eventos críticos.

5. Melhorias Futuras

Com base na implementação atual, as seguintes evoluções são sugeridas para tornar o sistema produtivo:

- **Integração com broker MQTT real (Mosquitto)**– substituir a simulação HTTP por comunicação assíncrona e leve, mais adequada para dispositivos IoT
- **E-mail real com smtplib**– substituir a simulação de e-mail por envio efetivo utilizando servidor SMTP com credenciais
- **Dashboard em tempo real**– utilizar WebSockets (Socket.IO) para atualização contínua dos gráficos sem necessidade de refresh
- **Banco de dados persistente**– migrar do armazenamento em memória para SQLite (local) ou MongoDB (nuvem), garantindo resiliência e escalabilidade

Essas melhorias transformariam o protótipo atual em uma solução pronta para implantação em ambientes clínicos reais.